

Archivos de datos

Archivo

Es un conjunto de datos estructurados en una colección de **registros** de igual tipo y estos a su vez se componen de **campos**. Tienen nombre y dirección.

Registro

Es cada uno de los componentes del archivo, poseen igual estructura para todos, almacenan información referente al tema general del archivo y se componen de **campos**, los cuales se relacionan entre sí.

Pueden ser de longitud:

- **Fija:** siempre tienen el mismo tamaño y son fáciles de manejar, no siempre se utiliza bien el espacio, ya que no todos los valores asignados llenan el espacio reservado.
- **Variable:** el tamaño varía.

Campo

Es la mínima unidad de información de un registro y contiene un valor único. Pueden ser de tamaño fijo o variable y se pueden dividir en **subcampos**. Tiene longitud, tipo de dato y tamaño fijo o variable.

Clave

Es un **campo** (o conjunto de ellos) que identifica un **registro** y facilita la extracción de información del mismo. Debe ser: **único** y **mínimo** (una clave por registro).

Tipos de archivos: según ciclo de vida

1. **Persistentes:** Se almacenan en memoria secundaria y deben eliminarse explícitamente.
2. **Temporales:** Pueden residir en memoria principal o secundaria y tienen un tiempo de vida determinado.

Tipos de archivos: según la información contenida

1. **De programa:** Almacenan instrucciones para manipular los datos.
2. **De datos:** Almacenan datos en forma de registros.
 - **Archivo maestro:** conjunto de registros acerca de un aspecto importante de las actividades de la organización. Son útiles si son exactos y se actualizan (mediante archivos de **transacciones**).
 - **Archivos de transacciones:** archivo temporal que registra eventos al ocurrir y actualiza archivos maestros con los resultados de las transacciones.
 - **Archivo de reportes:** archivo temporal que contiene reportes en cola de impresión. Se escribe en un archivo en disco, donde permanece hasta que pueda imprimirse.
 - **Archivo de respaldo:** copia de un archivo maestro o de transacciones.

Organización

Se organizan según la forma en que se estructuran en el almacenamiento.

Secuencial

Cada registro es adyacente al siguiente y el acceso es en orden de escritura. No existen posiciones sin uso. Las operaciones R&W avanzan el puntero al registro siguiente.

	Nombre	Edad	Estatura	IQ
1	Arias	55	5'8"	95
2	Benitez	39	5'6"	75
3	Bona	36	5'7"	70
4	Bresan	25	5'6"	49
5	Calderón	27	5'11"	80
6	Ferrer	42	5'9"	178
7	García	61	5'6"	169
8	Portal	36	5'7"	83
9	Puch	31	5'6"	95
10	Yañez	59	5'5"	145
11	Klein	26	5'8"	47
12	Miller	27	5'2"	75

Archivo Secuencial

D.N.I.	APELLIDO	NOMBRE	CIUDAD	...
23114589	Benítez	Claudia	Oro Verde	...
0	Miller	Carlos	Cerrito	...
27415777	Bona	Hernán	Viale	...
23066774	Bertolo	Pablo	Paraná	...
...	...	...	...	...

En su **alta** se agrega seguido al último dado de alta, la **baja** es lógica y para el **acceso** se utiliza una clave de búsqueda que es comparada con cada registro del archivo.

o Costo en fracaso = $n + 1$

o Costo promedio = $(n + 1) / 2$

Como **ventaja** destaca el aprovechamiento de memoria, sin embargo los datos en estos archivos son difíciles de combinar con otros.. Si se usa más del 40% de los registros, es recomendable su uso; si se usa menos del 10% no es recomendable y entre 10% y 40% depende del tamaño del archivo, frecuencia de uso, etc. Si se quiere hallar un registro particular en un archivo muy grande no es recomendable.

Se **usa** para procesamiento comercial de datos orientado al manejo por lotes y aplicaciones offline.

Indexado

La dirección relativa dentro del fichero se asocia a la clave de un registro mediante una *tabla de contenido*. Es conveniente cuando hay variación de volumen.

El **área de datos** almacena los registros y se direcciona por el área de índice.

El **área de índice** tiene la clave del registro y la dirección de almacenamiento asociada. Se implementa con un árbol B y es de constante actualización. Un archivo puede tener más de un índice y éste puede tener uno o varios campos claves.

El fichero indexado puede contener las claves de todos los registros o un subconjunto de ellas.

La **densidad del índice** es la división entre la cantidad de claves en el índice y la cantidad de registros del fichero.

- **Índice denso:** el índice contiene las claves de todos los registros.
- **Índice no denso:** el índice contiene menos claves que la cantidad de registros en el fichero. Los registros se almacenan ordenados en bloques que contienen el índice, la clave mayor y la dirección del bloque para cada bloque del archivo.

Para localizar un registro se debe:

1. Leer el índice.
2. Ubicar la clave.
3. Tomar la dirección.
4. Buscar el dato en el área de datos.

Los registros deben estar ordenados con respecto a su clave.

Indexado puro

El **área de datos** puede estar estructurada en espacios contiguos o cada registro estar distribuido de forma individual.

El **área de índice** almacena una clave y una dirección asociada a la clave por cada registro en el área de datos.

Una **clave** es uno o más *campos* que identifican unívocamente un registro. El **área de claves** se representa con un árbol B.

Área de Índice		Área de Datos			
Clave de Registro	Dirección				
AB	1021	1021	24613787	Alvarez	Andrea
AC	1021	1021	24592208	Torres	Diego
AD	1022	1022	84507114	Alvarez	José
BC	1018	1018	24300116	Silva	Mariana

Para el **alta**, la nueva clave se ubica en el *área de índice*, se consulta y reserva una dirección en el *área de datos*, asociándola al índice. Luego, se graba el registro en el área de datos reservada y se encadena con los registros existentes.

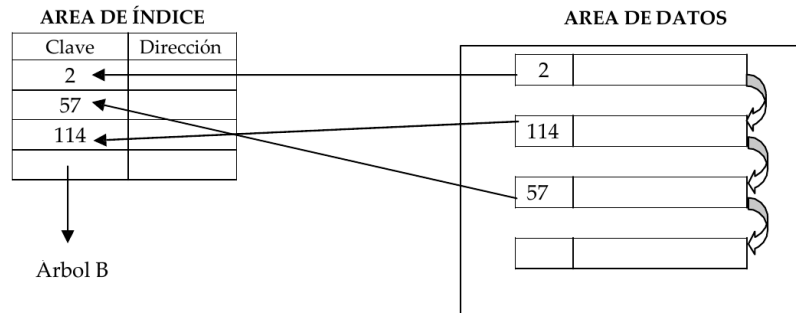
La **baja** puede ser *física* (área de índice) o *lógica* (área de datos). Hay dos formas de hacer una baja **física**:

- Se busca la clave, se obtiene la dirección y se realiza la baja física y lógica.
- Se ubica el registro en el área de datos, se libera el lugar que ocupaba y se elimina físicamente.

Para **modificar** se regrababa sobre el registro. Se ubica la clave (área de índice), se localiza el registro (área de datos) y este se carga en memoria para modificarse y grabarse.

Para el **acceso**, dada una clave se la busca en el área de índice y, si se encuentra, se busca la información en el área de datos.

Su utilización es para sistemas online o cuando los datos son muy variables y dinámicos.



Reconstrucción de un archivo indexado

Si se pierde el área de índice completa, no existe forma de acceder. Para reconstruir el archivo se usa un regenerador que va al área de datos.

Si el espacio es contiguo y la clave existe, el índice se crea con clave y dirección. Si la clave no está, no se genera el índice. Si el espacio no es contiguo, se necesita la dirección del primer al último registro y guardar la clave en el registro de datos para reparar el índice antes de generarlo.

Secuencial indexado

Los archivos se organizan por bloques de registros donde los índices apuntan a cada bloque. El índice apunta al primer elemento del bloque.

- **Área primaria de datos:** almacena los registros.
- **Área de overflow:** almacena los registros cuando el área primaria está llena.
- **Área de índice:** almacena los índices.

El **área primaria** se organiza en bloques (en orden secuencial). Dentro de cada bloque hay una cantidad fija de registros ordenados por clave. El acceso a cada *bloque* es directo y en el bloque se busca la clave secuencialmente.

El **área de índice** almacena las direcciones físicas del comienzo de cada bloque y su clave asociada. Es un índice por bloque.

El **área de overflow** almacena los registros cuando el área primaria está llena. Es de organización secuencial. La dimensión es un 15% a 20% del área de datos. Si este área crece mucho, el dimensionamiento no fue correcto.

AREA DE INDICE

Clave del registro	Dirección de comienzo del bloque
1115	1345
1315	1349
1429	1346
1725	1350

AREA DE DATOS

1345	1010	1011	1013	1014	1017	1019	1110	1113	1115					
1346	1316	1317*	1321	1323	1324	1410	1414	1415	1417	1418	1419	1427	1428	1429
1349	1117	1120	1121	1210	1211	1212	1215	1217	1218	1221	1310	1313	1315	
1350	1510	1521	1522	1617	1619	1620	1721	1724	1725					
O- F-	*1318													

El proceso de **alta** de un nuevo registro se gestiona de la siguiente manera:

1. Archivo Inexistente (Primer Alta):

- Se crea un nuevo bloque.
- El registro se graba con su clave en la primera posición del bloque.
- El área de índice se actualiza con la clave y la dirección de este nuevo bloque.

2. Archivo Existente (Altas Posteriores):

- **Determinación del Bloque:** Se busca el bloque donde debería ubicarse el nuevo registro (se recorre el índice hasta encontrar una clave mayor, indicando que el registro pertenece al bloque anterior).
- **Grabación en Bloque de Datos:**
 - Una vez determinada la dirección del bloque, el registro se graba al final del mismo.
 - Si hay espacio suficiente en el bloque de datos, el alta se realiza directamente.
- **Gestión del Desbordamiento (Overflow):**
 - Si el lugar requerido para la nueva clave excede el espacio previsto o el bloque está lleno, se pueden dar dos situaciones:
 - **Nuevo Bloque:** Si una clave es superior a la última del bloque actual (y se ha llenado el espacio restante), se genera y se abre un nuevo bloque, se inserta el registro y se actualiza el área de índice.
 - **Área de Overflow:** Si no hay lugar en el bloque de datos, el registro se graba en el *área de desborde*, actualizando los *links* (punteros) correspondientes.

Consideraciones Adicionales:

- **GAP de Protección:** Cada bloque incluye un espacio de protección (GAP) del 30-40%. Este espacio está destinado a la inserción de nuevos registros cuyas claves se encuentren entre la clave mínima del bloque actual y la clave mínima del bloque siguiente.

La **baja** puede ser lógica dentro del bloque o hacerse un corrimiento. Si el registro a borrar es el índice, se debe modificar el *área de índice*. Los bloques se llevan a memoria y se recorren.

Solo puede hacerse una baja física si el registro es único, donde desaparece el bloque y su referencia al área de índice.

Para **modificar** se puede regrabar sobre el registro. Si se cambia la clave, se debe reorganizar el área de índice.

Para **acceder** a un archivo se hace mediante un índice al bloque y dentro de este con una búsqueda secuencial.

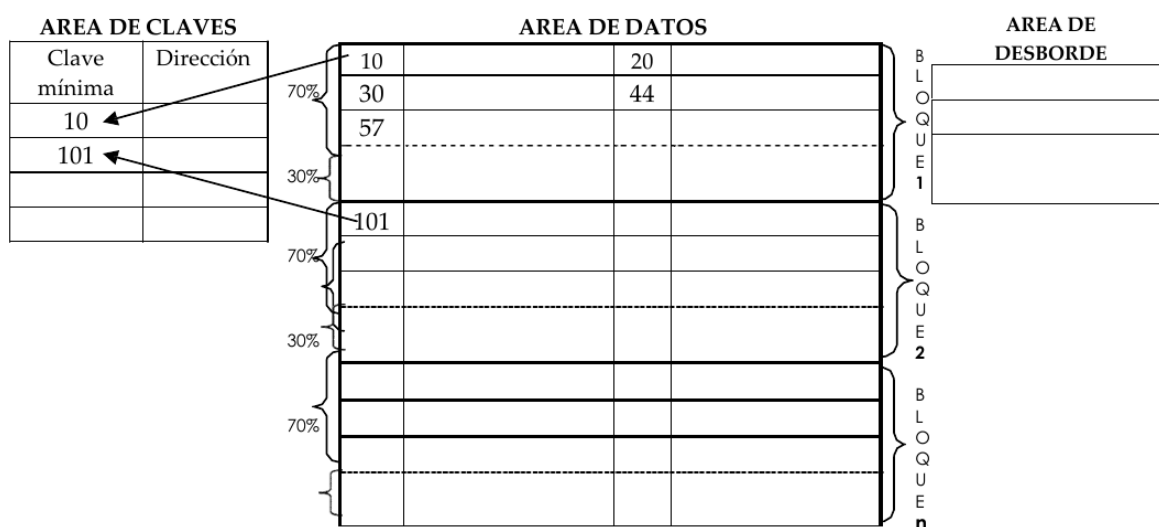
1. Acceder al índice.

2. Comparar la clave de búsqueda hasta ubicarla.
3. Compara la clave de cada registro con la clave de búsqueda tras leer los registros ubicados en la dirección asociada a la clave.

Se destacan como **ventajas** que el fichero está ordenado, lo que facilita el acceso secuencial en orden creciente. Además, equilibra el tamaño de la tabla de índices y el tiempo de acceso a los registros.

Como **desventaja** se menciona que se asigna espacio de almacenamiento que puede no ser utilizado en el bloque. Aparte, la gestión del área de overflow es compleja y degrada las prestaciones. Este área puede crecer excesivamente, lo que ralentiza y dificulta su procesamiento.

Estos archivos se utilizan para sistemas online de grandes volúmenes de datos, cuando se necesita conservar archivos actualizados de forma constante, para acceso orientado a terminales y para manejar consultas.



Directos

Referencia directamente a su posición relativa en el almacenamiento, por lo que son de acceso directo.

Previo a usarlo debe dimensionarse, y para esto se debe tener en cuenta la cantidad de registros a almacenar y la longitud de cada uno.

Cada registro tiene un **Número Relativo de Registro (NRR)** que no es la dirección física en el disco. Es conveniente que la clave del registro se relacione con el NRR para su acceso. No precisa estar ordenado y tiene un área de overflow.

Para el **alta** debe posicionarse según el NRR y grabar. Al acceder directamente, puede haber espacios entre los registros.

Las **bajas** son lógicas.

Para **modificar** se debe ubicar el registro, leerlo, llevarlo a memoria y modificarlo.

Para el **acceso** se calcula la dirección del bloque físico que lo contiene y se accede directamente. También puede accederse mediante la clave si está relacionada con el NRR. Un punto medio entre ambos es el hashing: se hasha el NRR. Si no se puede transformar una clave, se simula una lectura secuencial.

El acceso directo es rápido y si se puede asociar la clave con el NRR, es 100% eficiente. Si no se puede, no sirve y, en el medio de ambos, existe el hashing.

Este tipo de archivos se utiliza cuando la información no varía mucho, los registros son pequeños y fijos, el acceso rápido es esencial y el acceso a los datos es simple.

Clasificación: costo-beneficio

	Archivo Secuencial	Archivo Indexado	Archivo Secuencial Indexado	Archivo Directo
Uso de la memoria	Óptimo.	Utiliza más memoria que un archivo secuencial, pero es justificado ya que tiene un acceso bastante rápido.	Bueno.	Regular, ya que se debe dimensionar el archivo antes de crearlo, con lo que puedo llegar a malgastar memoria si sobran registros, o puede llegar a ser poca la cantidad de registros reservados.
Eficiencia en búsquedas y consultas	Es ineficiente, ya que se debe recorrer uno a uno todos los registros hasta encontrar el que se desea. El costo de las búsquedas y consultas es alto; para disminuirlo se sugiere ordenar los registros (en forma ascendente o descendente) en función de algún campo de los registros.	Son bastante eficientes, más rápido que el secuencial y que el secuencial indexado, pero más lento que el directo.	Buena, es más rápida que la secuencial, aunque más lenta que la directa.	Bastante eficiente, ya que posee costo de acceso 1.

Teoría de bases de datos

Definición

Colección de datos interrelacionados almacenados sin redundancias perjudiciales, independientes de los programas y con métodos determinados para su gestión. (J. Martin)

Los **datos** permanecen en el tiempo, son estructurados, íntegros, tienen sentido semántico y se manipulan utilizando operadores. Los que componen una base de datos se denominan **datos de operación** que son la información principal, viva y dinámica que una base de datos almacena para respaldar las actividades y transacciones diarias de una organización o sistema.

La **independencia de datos** es la capacidad de modificar la estructura de los datos o la forma en la que se almacenan, sin tener que alterar o reescribir los programas (procesos o aplicaciones) que los utilizan.

- **Independencia Física:** La capacidad de modificar el esquema interno (cambiar índices, dispositivos o técnicas de compresión) sin afectar el esquema conceptual ni los programas de aplicación.
- **Independencia Lógica:** La capacidad de modificar el esquema conceptual (añadir entidades o atributos) sin alterar los esquemas externos ni los programas existentes.

Características

1. **Integrada:** Unificación de archivos independientes eliminando redundancias parciales o totales.
2. **Compartida:** Permite que diferentes usuarios accedan a los mismos datos con propósitos distintos.

Esquema del diseño de la DB

1. **Diseño conceptual:** Es la representación total y abstracta de los datos que componen la DB y se representa con un **esquema conceptual**.
2. **Diseño lógico:** Se transforma el diseño conceptual al modelo de datos del SGBD.
3. **Diseño físico:** Se implementa el diseño lógico de forma eficiente. Depende del SGBD y el equipo donde se implemente.

Componentes de los sistemas de bases de datos

- La base de datos.
- SGBD.
- Programas de aplicación.
- Usuarios.
- Hardware.
- Programas utilitarios.

Arquitectura general

1. **Vista externa:** Es la visión particular de un usuario o grupo de ellos. Se define o formaliza con un **esquema externo**.
2. **Vista conceptual:** Es la representación total y abstracta de los datos que componen la base. Se define o formaliza con un **esquema conceptual**.
3. **Vista interna:** Es el nivel más bajo de la arquitectura y corresponde al almacenamiento físico de los datos de la base. Se define o formaliza con un **esquema interno**.

Construcción y definición de una DB

1. **Diseño conceptual:** Se establece conceptualmente la estructura de la DB. Para definirlo se utiliza el *DER*.
2. **Diseño lógico:** Se convierte el diseño conceptual siguiendo tres reglas:
 - a. Toda entidad se convierte en tabla.
 - b. Toda relación 1:n se convierte en una propagación de clave (primaria o foránea).
 - c. Toda relación n:m se convierte en una tabla intermedia.
3. **Diseño interno:** Depende del SGBD, donde se evalúan estrategias de almacenamiento, claves de acceso, técnicas de criptografía, etc.

Normalización: dependencias funcionales

Antes de aplicar la normalización, es fundamental entender cómo se relacionan los atributos entre sí mediante restricciones llamadas **dependencias**.

- **Dependencia Funcional ($X \rightarrow Y$):** Es una restricción que indica que los valores de un atributo (o conjunto de atributos) X determinan de manera única los valores de otro atributo Y .
- **Dependencia Funcional Plena:** Ocurre cuando un atributo Y depende de una clave completa X , y no de un subconjunto de ella. (Concepto clave para la 2FN).
- **Dependencia Transitiva:** Ocurre cuando un atributo A determina a B , y B determina a C ($A \rightarrow B$ y $B \rightarrow C$). Por lo tanto, C depende indirectamente de A . (Concepto clave para la 3FN).
- **Dependencia Multivaluada ($A \twoheadrightarrow B$):** Ocurre cuando para un valor de A existe un conjunto de múltiples valores válidos de B , y estos son independientes de los demás atributos.

Teoría de normalización

La **normalización** es el proceso mediante el cual se transforman datos complejos a un conjunto de estructuras de datos más pequeñas que, además de ser más simples y estables, son más fáciles de mantener. Su *objetivo* es mejorar el sistema lógico, de modo que, entre otros, se evite la duplicidad de datos.

1FN

Una relación está en *1FN* si *sus dominios no tienen elementos que a su vez sean conjuntos*, es decir, **cada atributo de una tabla contiene un valor atómico** (simple).

2FN

Una relación está en *2FN* si, además de estar en 1FN, *cualquiera de sus atributos no-primarios tiene una dependencia funcional plena con cada una de las claves candidatas de la relación*, es decir, **cada atributo no clave depende de la clave completa y no de parte de ella**.

3FN

Una relación está en *3FN* si, además de estar en 2FN, *ninguno de sus campos no-primarios tiene dependencias transitivas respecto de las claves candidatas de la relación*, es decir, ***todos los atributos no claves son independientes entre sí***.

FNBC (Forma Normal de Boyce-Codd)

Es una versión más fuerte y estricta que la 3FN. Una relación está en *FNBC* si, además de estar en 3FN, **todo determinante es una clave candidata**. Es decir, cualquier atributo (o grupo de atributos) del cual dependa otro, debe ser capaz de identificar unívocamente a toda la fila. Cubre ciertas anomalías que la 3FN no detecta cuando hay claves candidatas compuestas y superpuestas.

4FN

Una relación está en *4FN* si, además de estar en FNBC, **no posee dependencias multivaluadas** que no provengan de las claves candidatas. Asegura que atributos independientes con múltiples valores no estén mezclados en la misma tabla, evitando la repetición masiva de datos.

5FN

Una relación está en *5FN* si está en 4FN y **toda Dependencia de Join (reunión) viene implicada por las claves de la tabla**. Garantiza que si una tabla se descompone, pueda volver a unirse sin generar registros falsos o pérdida de información. Tiene poca aplicación práctica porque dificulta el diseño al generar un número enorme de tablas.

Sistema gestor de bases de datos

Es un conjunto de programas, procedimientos y lenguajes que da a los usuarios los medios para describir, recuperar y manipular los datos almacenados en la DB, manteniendo su integridad, confidencialidad y seguridad.

Funciones del SGBD

- **Función de Definición:** Da al DBA herramientas para que especifique la información diseñada en el **esquema conceptual**, provee un **lenguaje de definición de datos (LDD/DDL)** y debe ser capaz de definir las estructuras de datos a nivel *externo, conceptual e interno*.
 - A **nivel interno** se definen el espacio reservado, la longitud de los campos, como se representan los datos y su forma de acceso.
 - A nivel **externo y conceptual** se dan las herramientas para definir las entidades, su identificación, atributos, mapeo, etc.
- **Función de Manipulación:** Realizada mediante el **Lenguaje de Manipulación de Datos (LMD/DML)**. Permite buscar, eliminar, modificar y añadir datos. Puede ser procedimental (especifica cómo acceder) o no procedimental (especifica qué obtener, como SQL).
- **Función de Utilización (Servicio):** Proporciona interfaces para el DBA para control de acceso, auditoría, respaldos (backup), carga/descarga y estadísticas de utilización. Además, supervisa las solicitudes de usuario y rechaza intentos de violar medidas de seguridad e integridad, asegura coherencia de los datos luego de ser manipulados y presenta el diccionario de datos.

Lenguaje de definición de datos (LDD/DDL)

Es una parte del **sublenguaje de datos (SLD/DSL)** del SGBD encargada de la función de definición. Se utiliza para describir los elementos de los datos, sus estructuras, interrelaciones y validaciones a nivel externo, lógico e interno. En pocas palabras, es el lenguaje que usas para especificar el esquema conceptual y crear las estructuras ("el molde") de la base de datos.

Lenguaje de manipulación de datos (LMD/DML)

Es la otra parte del **sublenguaje de datos (SLD/DSL)** del SGBD encargada de la función de manipulación de la información. Se utiliza exclusivamente para interactuar con los datos ya almacenados: buscar (consultar), añadir (insertar), suprimir (eliminar) y modificar los datos de la base de datos.

Componentes del SGBD

- **Procesador de consultas:** Traduce las sentencias a instrucciones de bajo nivel.
- **Gestor de archivos:** Gestiona la asignación de espacio en disco y de las estructuras de datos.
- **Precompilador de DML/LMD:** Convierte las sentencias en DML/LMD.
- **Compilador de DDL/LDD:** Convierte las sentencias en DDL/LDD en un conjunto de tablas metadatos.
- **Gestor del diccionario de datos:** Almacena metadatos sobre la estructura de la DB..
- **Gestor de la DB:** Interfaz entre los datos de bajo nivel, los programas y las consultas. Sus componentes son:

- **Control de autorización:** Comprueba que el usuario tiene los permisos para llevar a cabo la operación que solicita.
- **Procesador de comandos.**
- **Control de integridad:** Comprueba que la operación a realizar satisface las restricciones de integridad.
- **Optimizador de consultas:** Optimiza la ejecución de consultas.
- Gestor de transacciones: Procesa las transacciones.
- **Planificador:** Asegura que las operaciones que se realizan en simultáneo no tienen conflictos.
- **Gestor de recuperación:** Garantiza la consistencia en caso de fallas.
- **Gestor de buffer/datos:** Transfiere los datos entre la memoria principal y los dispositivos de almacenamiento secundario.

Clasificación de los SGBD

- Según el modelo lógico puede ser jerárquico, en red, relacional u orientado a objetos.
- Según el número de usuarios puede ser monousuario o multiusuario.
- Según el número de sitios puede ser centralizado o distribuido.
- Según el ámbito de la aplicación puede ser general o específico.

El Rol del Administrador de Bases de Datos (DBA)

Es el responsable del control general del sistema. Sus funciones incluyen:

1. Decidir el contenido de la BD (Modelo Conceptual).
2. Definir la estructura de almacenamiento y estrategia de acceso (Modelo Interno).
3. Vincularse con los usuarios para garantizar esquemas externos.
4. Definir controles de autorización, seguridad y validación.
5. Establecer estrategias de respaldo y recuperación.
6. Monitorear el desempeño y responder a cambios de requerimientos.

DER

Un **diagrama entidad-relación** es una herramienta visual utilizada en la fase de diseño conceptual para modelar la estructura de una base de datos.

Características principales:

- **Entidades:** Representan objetos o conceptos.
- **Atributos:** Son las propiedades de esas entidades. Tienen un **identificador**, que es un atributo con unicidad y minimalidad que distingue a cada registro.
- **Relaciones:** Definen cómo interactúan las entidades entre sí, mediadas por reglas de cardinalidad.

Pasos para armar un DER básico:

1. Identificar los tipos de entidad y las relaciones que existen entre ellos.
2. Identificar los atributos de cada elemento.
3. Definir los identificadores principales.
4. Establecer las cardinalidades y restricciones.

Características del DER Extendido

Sirve para modelar escenarios más complejos mediante:

- **Atributos especiales:** Compuestos (se dividen en partes, como "Dirección") o Multivaluados (tienen varios valores, como "Teléfonos").
- **Entidades débiles:** Su existencia y clave dependen de una entidad fuerte (ej. un "Ejemplar" depende de un "Libro").
- **Generalización:** Permite crear jerarquías (ej. "Persona" abarca a "Profesor" y "Alumno"), definiendo si la cobertura es total/parcial o exclusiva/superpuesta.
- **Agregación:** Trata una relación entera como si fuera una entidad para poder relacionarla con una tercera entidad.

MR

El **modelo relacional** es un enfoque lógico para la gestión y organización de datos en una base de datos que organiza la información estructurándola en los siguientes elementos:

- **Tablas (Relaciones):** Los datos se almacenan en una colección de tablas bidimensionales.
- **Filas (Tuplas o Registros):** Cada fila dentro de una tabla representa un conjunto de datos único sobre una entidad específica.
- **Columnas (Atributos o Campos):** Cada columna describe una propiedad particular o un dato específico que conforma el registro.
- **Claves (Keys):** Se utilizan identificadores únicos (**claves primarias**) para distinguir cada fila, y se establecen vínculos entre diferentes tablas utilizando referencias cruzadas (**claves foráneas**).

Su principal ventaja es que garantiza la integridad, consistencia e independencia de los datos, permitiendo relacionar información de distintas tablas y realizar consultas complejas, generalmente utilizando el lenguaje estandarizado **SQL**.

Reglas para pasar al Modelo Relacional (Tablas):

- **Entidades y atributos:** Cada entidad se convierte en una tabla y sus atributos simples en columnas. Los atributos **compuestos** se separan en varias columnas (calle, número, piso) y los **multivaluados** se separan en una tabla nueva.
- **Relaciones 1:1 o 1:N:** La clave principal de una entidad se propaga a la otra tabla para vincularlas.
- **Relaciones N:M:** Se debe crear una nueva tabla intermedia que contenga las claves principales de las dos entidades involucradas y los atributos de la interrelación.
- **Entidad Débil:** Se transforma en una tabla donde su clave se compone sumando la clave de su entidad fuerte.
- **Generalización:** La forma recomendada es crear una tabla para la entidad general (ej. Persona) y tablas separadas para los subtipos (ej. Empleado, Cliente), unidas por la clave principal.